

Assignment 6

GUI-Based Multi-Client Chat Application Using TCP

Name	Gouriniban Borphukan	Roll No.	CSB24032
------	----------------------	----------	----------

1. Objective

Convert the terminal-based TCP chat application from Assignment 5 into a graphical desktop application using Python's *tkinter* library. The objective is to introduce GUI programming, event-driven design, multithreading, and user-friendly network application development, while reusing the existing server and socket communication logic.

2. GUI Design

The application consists of two windows: a **Login Window** that collects the server IP and username before establishing the TCP connection, and a **Chat Window** that provides the full messaging interface. A dark-themed color palette was chosen for readability and to visually distinguish message types (broadcast, private, server notices).

2.1 Login Window (Task 1)

Contains a Server IP entry field, a Username entry field, a status label for validation feedback, and a Connect button. Input is validated — empty usernames, single-character names, and names with spaces are rejected with an inline error message. Pressing Enter also triggers the connect action.

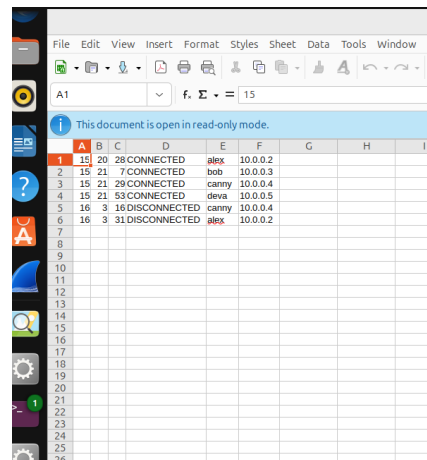
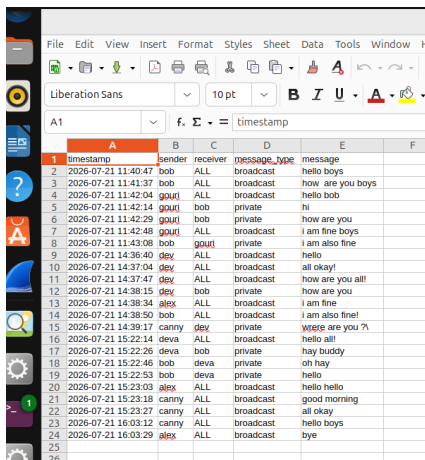


Fig 1. Login Window (left) and Chat Window (right)

2.2 Chat Window (Task 2)

The main chat window contains: a **scrollable message area** (color-coded by type), a **Private To** entry for targeting a specific user (empty = broadcast), a **message input box** with Send button and keyboard shortcut (Enter), a **Disconnect** button, an **Online Users** panel on the right (click to PM), a **commands reference** sidebar (/list, /stats, /quit), and a **status bar** showing connection details.

3. System Architecture

Layer	Class / Component	Responsibility
Networking	ChatClient	TCP socket, connect, send, recv loop, disconnect
Background I/O	_recv_loop (daemon thread)	Receives messages — never blocks GUI
GUI – Login	LoginWindow	Input validation, connection initiation

GUI – Chat	ChatWindow	Message display, user list, send controls
Entry Point	App	Creates root Tk window, switches Login → Chat
Server	server.py (reused)	Accepts clients, routes messages, logs events

The **ChatClient** class is completely decoupled from the GUI — it exposes *connect()*, *send()*, *send_pm()*, *disconnect()*, and *start_receiver()*. The receiver thread calls back into the GUI via *root.after(0, ...)* to safely update tkinter widgets from a non-main thread.

4. Components Reused from Assignment 5

The server (**server.py**) is reused entirely without modification. All networking protocol elements designed in Assignment 5 are preserved in the GUI client:

- TCP socket connection on port 5000 with *ENTER_USERNAME* handshake
- Broadcast messaging — plain text sent to server, echoed to all peers
- Private messaging — */msg <target> <text>* protocol
- */list* and */stats* command protocol
- */quit* graceful disconnection
- Chat history replay on reconnect (last 5 messages shown in welcome)
- Server-side logging to **server_log.csv** and **chat_history.csv**

The only change is the **client side**: the terminal *input()* / *print()* loop is replaced by the tkinter GUI, while the *ChatClient* networking class preserves identical socket logic.

5. GUI Design Decisions

Separation of concerns: *ChatClient* is a pure networking class with no tkinter imports. *LoginWindow* and *ChatWindow* are pure GUI classes that call *ChatClient* methods. This mirrors the MVC pattern and makes each layer testable independently.

Background receiver thread: *socket.recv()* is blocking — running it in the main GUI thread would freeze the window. A daemon thread runs *_recv_loop()* and posts updates back to tkinter via *root.after(0, callback)*, the only thread-safe tkinter update mechanism.

Color-coded messages: Server notices (yellow/italic), own messages (blue/bold), peer messages (white), incoming PMs (green/bold), outgoing PMs (purple/bold). Immediate visual distinction reduces cognitive load in a multi-user session.

Click-to-PM user list: Clicking a name in the Online Users listbox auto-fills the Private To field, removing the need to type usernames manually and reducing errors.

In-place window switching: App reuses the same Tk root; *LoginWindow* widgets are destroyed and replaced by *ChatWindow* on successful connect, avoiding a second top-level window.

6. Testing Results

Four clients (alex, bob, canny, deva) connected simultaneously inside a Mininet single-switch topology (*sudo mn --topo single,5*). The test session covered all required features.

Test	Expected	Result
Login — empty username	Error: cannot be empty	✓ Shown
Login — valid username	Chat window opens	✓ Opens
Broadcast message	All clients receive [sender] msg	✓ Delivered
Private message (/msg)	Only target receives [PM from]	✓ Routed correctly
Join notification	"X has joined" shown to all	✓ Broadcast
Leave notification	"X has left" shown + removed from list	✓ Broadcast + UI
Online user list update	Listbox adds/removes on join/leave	✓ Auto-updated
Click-to-PM	Clicking name fills Private To field	✓ Works

Disconnect button	Sends /quit, closes window	✓ Graceful
Chat history replay	Last 5 messages shown on reconnect	✓ Shown

Log Evidence

The screenshots below show **chat_history.csv** confirming broadcast and private message logging with timestamps, and **server_log.csv** confirming CONNECTED / DISCONNECTED events for all four clients.

	A	B	C	D	E	F
1	timestamp	sender	receiver	message_type	message	
2	2026-07-21 11:40:47	bob	ALL	broadcast	hello boys	
3	2026-07-21 11:41:37	bob	ALL	broadcast	how are you boys	
4	2026-07-21 11:42:04	gaut	ALL	broadcast	hello bob	
5	2026-07-21 11:42:14	gaut	bob	private	hi	
6	2026-07-21 11:42:29	gaut	bob	private	how are you	
7	2026-07-21 11:42:48	gaut	ALL	broadcast	I am fine boys	
8	2026-07-21 11:43:08	bob	gaut	private	I am also fine	
9	2026-07-21 14:36:40	dev	ALL	broadcast	hello	
10	2026-07-21 14:37:04	dev	ALL	broadcast	all okay!	
11	2026-07-21 14:37:47	dev	ALL	broadcast	how are you all!	
12	2026-07-21 14:38:15	dev	bob	private	how are you	
13	2026-07-21 14:38:34	alex	ALL	broadcast	I am fine	
14	2026-07-21 14:38:50	bob	ALL	broadcast	I am also fine!	
15	2026-07-21 14:39:17	canny	dev	private	Where are you ?	
16	2026-07-21 15:22:14	deva	ALL	broadcast	hello all	
17	2026-07-21 15:22:26	deva	bob	private	hey buddy	
18	2026-07-21 15:22:46	bob	deva	private	oh hey	
19	2026-07-21 15:22:53	bob	deva	private	hello	
20	2026-07-21 15:23:03	alex	ALL	broadcast	hello hello	
21	2026-07-21 15:23:18	canny	ALL	broadcast	good morning	
22	2026-07-21 15:23:27	canny	ALL	broadcast	all okay	
23	2026-07-21 16:03:12	canny	ALL	broadcast	hello boys	
24	2026-07-21 16:03:29	alex	ALL	broadcast	bye	
25						
26						

Fig 2. chat_history.csv — broadcast and private messages logged with sender, receiver, type

	A	B	C	D	E	F	G	H	I
1									
2	15:20	DISCONNECTED	alex	10.0.0.2					
3	15:21	7CONNECTED	bob	10.0.0.3					
4	15:21	DISCONNECTED	canny	10.0.0.4					
5	15:21	DISCONNECTED	deva	10.0.0.5					
6	16:03	DISCONNECTED	canny	10.0.0.4					
7									
8									
9									
10									
11									
12									
13									
14									
15									
16									
17									
18									
19									
20									
21									
22									
23									
24									
25									
26									

Fig 3. server_log.csv — CONNECTED / DISCONNECTED events for all four clients

7. Wireshark Verification

Traffic was captured inside Mininet using Wireshark on the virtual interface. Filter used: `tcp.port == 5000`. Four packet-level events were verified.

7.1 Client Connection (TCP Handshake + Username Registration)

SYN → SYN-ACK → ACK completes the three-way handshake. The Follow TCP Stream shows the ENTER_USERNAME prompt, username response, and the welcome/history replay from the server immediately after connection.

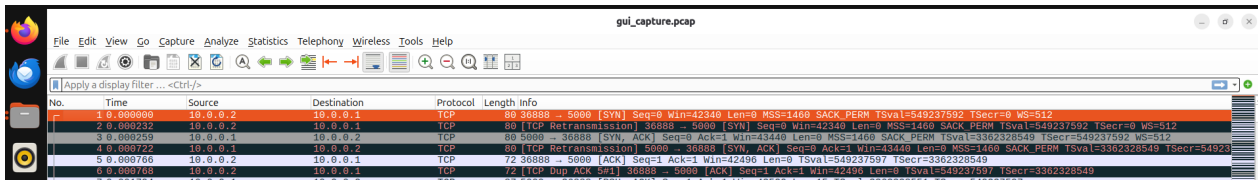


Fig 4. ws_client_connection.png — SYN/SYN-ACK handshake packets

7.2 Broadcast Message

The Follow TCP Stream (stream 1, bob's session) shows broadcast messages from other clients being delivered: [alex], [canny], [deva] messages all arrive on bob's connection, confirming server-side fan-out.

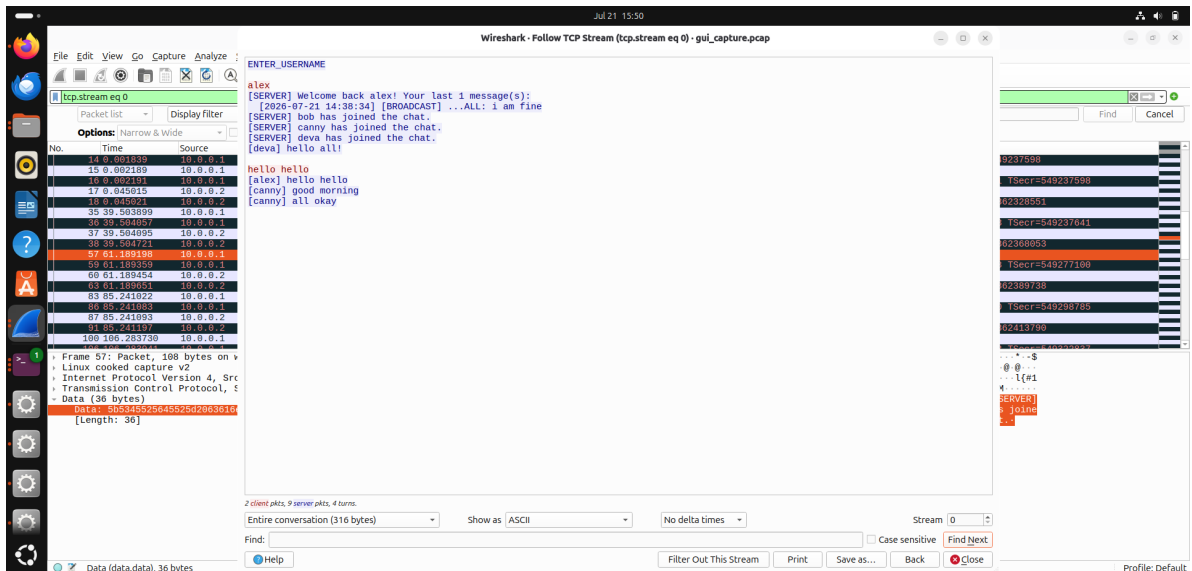


Fig 5. ws_broadcast.png — TCP stream showing broadcast delivery to bob

7.3 Private Message

The stream shows `/msg deva oh hay` sent by bob and the server routing `[PM to deva] oh hay` back as echo, and `[PM from bob] oh hay` delivered exclusively to deva's connection — confirming no broadcast leak.

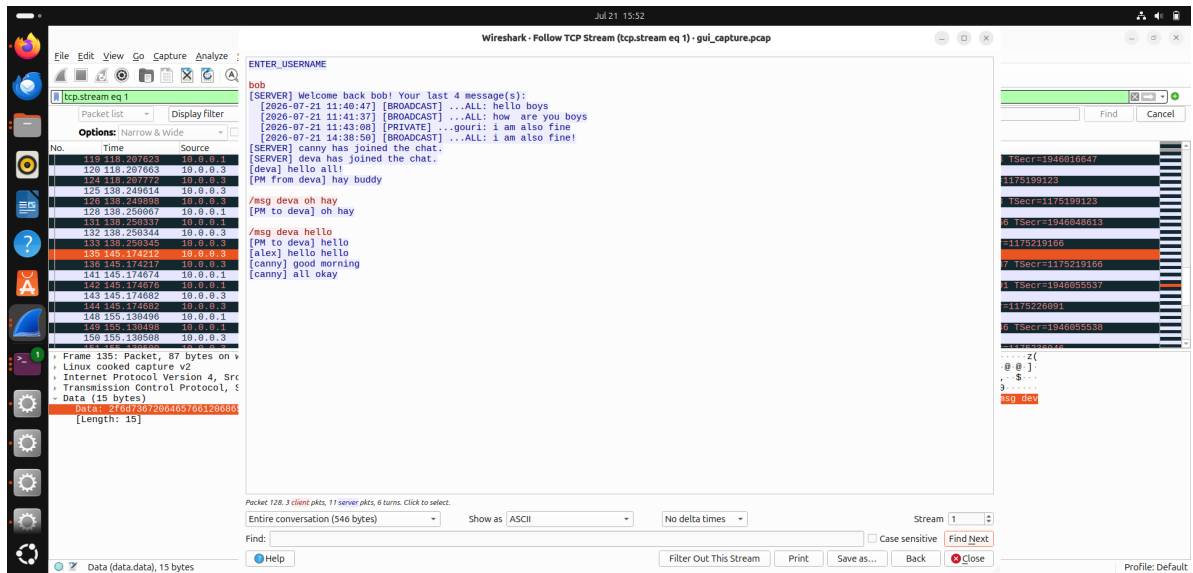


Fig 6. ws_private_msg.png — private message exchange visible in TCP stream

7.4 Client Disconnection (FIN/ACK)

FIN, ACK packets from both client and server sides confirm graceful TCP teardown when clients disconnect. The filter `tcp.flags.fin == 1` isolates these frames.

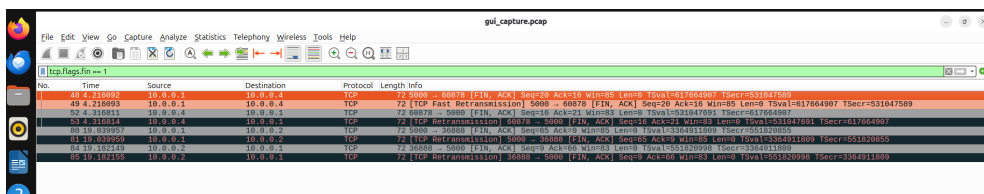


Fig 7. ws_disconnect.png — FIN,ACK frames for client disconnection

8. Reflection Answers

Q1. Why should networking logic and GUI code be separated?

Mixing them creates tight coupling — a `socket.recv()` call inside the main thread freezes the entire GUI until data arrives. Separation also enables independent testing: `ChatClient` can be unit-tested without a display server, and the GUI can be redesigned without touching the network protocol. It follows the single responsibility principle: one class owns communication, another owns presentation.

Q2. Why is a background thread required?

Tkinter's event loop runs on the main thread. Any blocking operation — particularly `socket.recv()` — stalls the event loop, preventing redraws, button responses, and text input. A daemon thread runs the `recv` loop independently; it posts results back via `root.after(0, callback)`, which queues the update safely on the main thread.

Q3. Which components from Assignment 5 were reused?

The entire `server.py` is reused unchanged. The client preserves the `ENTER_USERNAME` handshake, `/msg` private messaging protocol, `/quit` disconnection, `/list` and `/stats` commands, and the chat history replay on reconnect. Only the terminal I/O is replaced by the tkinter GUI.

Q4. How did the GUI improve usability?

Color-coded messages eliminate the need to parse prefixes manually. The online user panel gives instant visibility of who is present. Click-to-PM removes typing errors for private messages. The scrollable area retains full history, unlike a terminal that scrolls away. The status bar and connection indicator give persistent feedback on connection state.

Q5. Suggest one future enhancement.

File sharing: add a "Attach" button that reads a file, base64-encodes it, and sends it as a specially prefixed message. The receiver detects the prefix, decodes it, and saves it locally — enabling document sharing without leaving the chat application.

9. Conclusion

A fully functional GUI-based TCP chat application was developed by wrapping the Assignment 5 server and socket protocol with a tkinter desktop interface. The *ChatClient* networking class is completely decoupled from the GUI, with a background daemon thread handling all blocking I/O. The application supports user login with validation, broadcast messaging, private messaging via `/msg`, a live online user list with click-to-PM, color-coded message display, and graceful disconnect. All features were verified with four simultaneous Mininet clients, confirmed via server/chat logs and Wireshark packet capture. The GUI significantly improves usability over the terminal client while reusing the entire server implementation without modification.